



```
import pandas as pd

# Paste your raw GitHub URL here
url = 'https://raw.githubusercontent.com/SMont84/ITAI-1371-Midterm-GitGat/refs/heads/main/data/ITAI-1371-Midterm-GitGat.csv'

df = pd.read_csv(url)
df.head()
```

|   | id    | gender | age  | hypertension | heart_disease | ever_married | work_type     |
|---|-------|--------|------|--------------|---------------|--------------|---------------|
| 0 | 9046  | Male   | 67.0 | 0            | 1             | Yes          | Private       |
| 1 | 51676 | Female | 61.0 | 0            | 0             | Yes          | Self-employed |
| 2 | 31112 | Male   | 80.0 | 0            | 1             | Yes          | Private       |
| 3 | 60182 | Female | 49.0 | 0            | 0             | Yes          | Private       |
| 4 | 1665  | Female | 79.0 | 1            | 0             | Yes          | Self-employed |

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
!pip install -q imbalanced-learn
```

```
import pandas as pd
from sklearn.model_selection import train_test_split
from imblearn.over_sampling import SMOTE
from sklearn.preprocessing import LabelEncoder

target_column = 'stroke'

# 1. Convert target text to numbers if it's text
le = LabelEncoder()
df[target_column] = le.fit_transform(df[target_column])

# 2. Separate X and y
X = df.drop(columns=[target_column])
y = df[target_column]

# 3. Convert feature text columns to numbers
X = pd.get_dummies(X)

# 4. Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

'

```
print(df.isna().sum())
```

```
id                0
gender            0
age              0
hypertension      0
heart_disease     0
ever_married      0
work_type         0
Residence_type    0
avg_glucose_level 0
bmi              201
smoking_status    0
stroke            0
dtype: int64
```

```
df = df.fillna(df.mean(numeric_only=True))
```

```
print(X.isna().sum())
```

```
id                0
age              0
hypertension      0
heart_disease     0
avg_glucose_level 0
bmi              201
gender_Female     0
gender_Male       0
gender_Other      0
ever_married_No   0
ever_married_Yes  0
work_type_Govt_job 0
work_type_Never_worked 0
work_type_Private 0
work_type_Self-employed 0
work_type_children 0
Residence_type_Rural 0
Residence_type_Urban 0
smoking_status_Unknown 0
smoking_status_formerly smoked 0
smoking_status_never smoked 0
smoking_status_smokes 0
dtype: int64
```

```
import numpy as np
```

```
# Count how many infinite values exist in each column
print(np.isinf(X).sum())
```

```
id                0
```

```
age                                0
hypertension                       0
heart_disease                     0
avg_glucose_level                  0
bmi                                0
gender_Female                      0
gender_Male                        0
gender_Other                       0
ever_married_No                    0
ever_married_Yes                   0
work_type_Govt_job                 0
work_type_Never_worked             0
work_type_Private                  0
work_type_Self-employed            0
work_type_children                 0
Residence_type_Rural              0
Residence_type_Urban              0
smoking_status_Unknown            0
smoking_status_formerly smoked    0
smoking_status_never smoked       0
smoking_status_smokes             0
dtype: int64
```

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy='mean')
X_train_imputed = imputer.fit_transform(X_train)
X_test_imputed = imputer.transform(X_test)

smote = SMOTE(random_state=42)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train_imputed, y

# 10. Verify the results
print("Original Training Shape:", X_train_imputed.shape)
print("Resampled Training Shape (After SMOTE):", X_train_resampled.shape)
print("Success! SMOTE completed.")
```

```
Original Training Shape: (4088, 22)
Resampled Training Shape (After SMOTE): (7778, 22)
Success! SMOTE completed.
```

